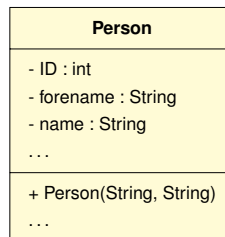


Exercice 1 : Encapsulation pour l'extension

Q1. Intéressons-nous d'abord à une personne.

Q1.1. Écrivez une classe `Person`, caractérisée par un numéro d'identification, un nom et un prénom. Fournissez à cette classe un constructeur spécifié par un prénom et un nom. L'identifiant est géré par un numéro automatique.



Q1.2. Écrivez tous les accesseurs et les mutateurs nécessaires afin de manipuler cet objet en sachant que la modification du numéro d'identification doit demeurer impossible.

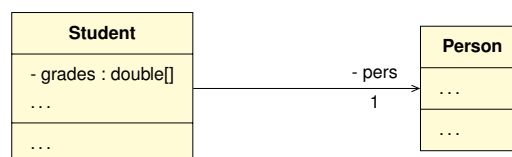
Q1.3. Ajoutez une méthode `toString`, qui retourne une instance sous la forme "`<id>: <forename> <name>`", et une méthode `equals`, en considérant que deux personnes sont identiques si elles ont le même numéro d'identification, peu importe leur nom et leur prénom.

```
String toString()
boolean equals(Object)
```

Q1.4. Récupérez la classe `UsePerson` sous Moodle et intégrez-la à votre projet. Exécutez-la pour déterminer si votre code fonctionne correctement.

Q2. On souhaite maintenant disposer d'une classe `Student` qui dispose des mêmes caractéristiques qu'une personne (`Person`), un tableau de notes en plus. Procédons par encapsulation. On supposera qu'il n'y a jamais de cases vides dans ce tableau.

Q2.1. Déclarez cette classe, qui doit répondre aux spécifications UML suivantes :



Q2.2. Munissez cette classe des constructeurs suivants :

```
Student(Person pers, double[] grades)
Student(String forename, String name, double[] grades)
Student(String forename, String name, double grade)
```

Un de ces constructeurs est certes correct, mais interdit dès lors qu'on souhaite encapsuler `Person`. Lequel ? Pourquoi ? Restreignez sa visibilité à `private`. N'oubliez pas d'utiliser le chaînage lorsque c'est possible.

Q2.3. Écrivez les méthodes déléguées, permettant la manipulation des attributs de la classe encapsulée.

Q2.4. Écrivez une méthode `toString`, retournant un étudiant sous la forme "`Student[<id>: <forename> <name> = <grades>]`". On pourra par exemple obtenir "`Student [1: Karim El Hadj = [12.5, 13.9]]`". Ajoutez également une méthode `equals` qui se base sur tous les champs dans la comparaison (jusqu'à la moindre note). N'oubliez pas d'invoquer les méthodes `toString` ou `equals` de la classe encapsulée, voire d'utiliser la boîte à outils `Arrays`.

```
String toString()
boolean equals(Object obj)
```

Q2.5. Ajoutez une méthode `getAverage` permettant le calcul de la moyenne d'un étudiant. Pour simplifier, un étudiant sans note obtient une moyenne de 0.

```
double getAverage()
```

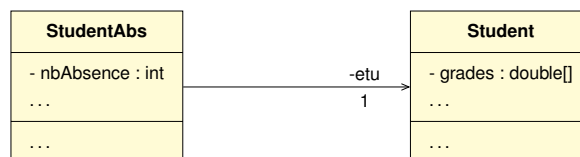
Q2.6. Ajoutez enfin une méthode `addGrade` permettant l'ajout d'une nouvelle note à l'étudiant. Le tableau de notes n'est supposé contenir aucune case vide (il est de taille minimale nécessaire).

```
void addGrade(double aGrade)
```

Q2.7. Écrivez une classe `UseStudent` pour créer trois instances d'étudiants, vérifier s'ils sont égaux deux à deux et les afficher.

Q3. On souhaite enrichir cette classe `Student` à la gestion des absences. Pour ce faire, inutile de modifier cette classe, on va l'encapsuler et l'enrichir au sein d'une autre classe (comme pour `Student` vis-à-vis de `Person`).

Q3.1. Écrivez une classe `StudentAbs`, caractérisée par un étudiant et un nombre d'absences. Munissez cette classe des méthodes déléguées possibles.



Q3.2. Ajoutez les méthodes `toString`, qui retourne une instance de `StudentAbs` sous la forme "<Student>, nbAbs=<nbAbsence>", et `equals` à cette classe (faisant appel respectivement aux méthodes `toString` et `equals` de la classe encapsulée).

```
String toString()
boolean equals(Object obj)
```

Q3.3. Écrivez une méthode `warning` qui détermine si l'étudiant doit recevoir un avertissement, au regard de seuils passés en paramètres. Un avertissement est envoyé si son nombre d'absences est supérieur ou égal à `thresholdAbs` OU si sa moyenne est inférieure ou égale à `thresholdAvg`.

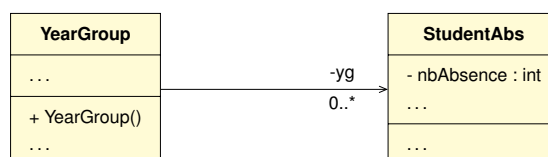
```
boolean warning(int thresholdAbs, double thresholdAvg)
```

Q3.4. Définissez de la façon analogue une méthode `validation` qui détermine si l'étudiant valide son année en fonction de son nombre d'absences et de sa moyenne, par rapport à des seuils passés en paramètre. Pour valider son année, un étudiant doit avoir strictement moins d'absences que `thresholdAbs` et une moyenne strictement supérieure à `thresholdAvg`.

```
boolean validation(int thresholdAbs, double thresholdAvg)
```

Q4. Intéressons-nous maintenant à une promotion d'étudiants.

Q4.1. Écrivez une classe `YearGroup` regroupant un ensemble d'étudiants, caractérisée uniquement par une liste d'étudiants, avec un constructeur sans paramètre. Ajoutez aussi l'accessor et le mutateur associés. On remarquera au passage qu'il ne s'agit plus ici d'encapsulation. . .



Q4.2. Ajoutez les méthodes `addStudent` qui ajoutent respectivement un étudiant donné et une liste d'étudiants donnée à la promotion.

```
void addStudent(StudentAbs other)
void addStudent(ArrayList<StudentAbs> others)
```

Q4.3. Munissez cette classe d'une méthode `addGrades` qui répartit le résultat d'une évaluation à chaque étudiant. On supposera que les notes sont fournies dans le même ordre que les étudiants dans la promotion et qu'il n'y a aucun absent.

```
void addGrades(double[] aTest)
```

Q4.4. Écrivez finalement une méthode `validation`, affichant tous les étudiants ayant validé leur année.

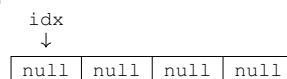
```
void validation(int thresholdAbs, int thresholdAvg)
```

Q4.5. Écrivez une classe `UseYearGroup` qui illustrent le bon fonctionnement de vos méthodes. Elle doit contenir au moins 4 étudiants A, B, C et D. A et B satisfont le critère d'assiduité. A et C satisfont le critère de moyenne. D ne satisfait aucun des critères.

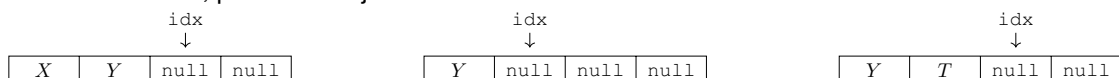
Q4.6. Réalisez un `commit` de votre travail associé au message : "tp00-04::exo-extension".

Exercice 2 : Encapsulation pour le contrôle d'accès

Le but de cet exercice est d'encapsuler un tableau en redéfinissant les méthodes de manipulation afin d'obtenir une file d'attente. Une telle file d'attente est basée sur un tableau dont les cases sont toutes initialisées à `null`, sur une capacité maximale pour la file et sur un indice `idx` indiquant où la prochaine insertion peut avoir lieu. Voici un exemple de file d'attente de capacité maximale 4, juste après son initialisation :



À chaque élément ajouté, on décale l'indice `idx`. À chaque élément retiré, on décale tous ceux encore présent (s'il y en a) et on décale l'indice dans l'autre sens. Voici un exemple basé sur une file de capacité 4, contenant de deux éléments, dont on en retire un, puis on en ajoute un autre.



Q1. Intégrez l'archive `tp04-res.jar` à votre projet. Cette archive contient le fichier binaire de la classe `PendingCase`. Cette classe a été initialement développée dans le paquetage `tp00.tp04`. Consultez également la documentation fournie pour déterminer comment manipuler une instance de cette classe (contenue dans le fichier `tp04-doc.zip`, à ouvrir via le fichier `index.html`).

Q2. Écrivez la classe `PendingCaseQueue` représentant une file de dossiers en attente de traitement (comme des piles de factures dont vous devez réclamer le paiement par exemple). Les files d'attente sont initialisées à partir d'une taille commune à toutes les instances, définie par défaut à 10. Cette classe doit être munie d'un constructeur sans paramètre, ainsi que d'une méthode `clear` vidant la file d'attente. Cette classe doit répondre à la structure suivante :

PendingCaseQueue
+ capacity : int
- idx : int
...
+ PendingCaseQueue()
+ clear() : void
...

Q3. Écrivez les méthodes `isEmpty` et `isFull` déterminant respectivement si la file est vide ou pleine, et qui renvoient des booléens.

```
boolean isEmpty()
boolean isFull()
```

Q4. Ajoutez la méthode `size`, donnant le nombre de dossiers en attente dans la file.

```
int size()
```

Q5. Écrivez une méthode `addOne` qui ajoute, lorsque c'est possible, un nouveau dossier dans la file d'attente.

```
void addOne(PendingCase other)
```

Q6. Écrivez la méthode `removeOne` qui retire et renvoie le premier dossier de la file, ou `null` le cas échéant. N'hésitez pas à décomposer votre code.

```
PendingCase removeOne()
```

Q7. Écrivez la méthode `toString` qui renvoie une file sous forme textuelle. La méthode doit afficher les dossiers dans l'ordre.

```
public String toString()
```

Q8. Récupérez la classe `UsePendingCaseQueue` munie d'une méthode `main` dans laquelle on instancie une file de capacité maximale 3 et 4 dossiers en attente. Utilisez-la pour contrôler le bon fonctionnement de vos méthodes méthodes `clear`, `addOne`, `isEmpty` et `isFull` face aux différents cas possibles.

Q9. Écrivez la méthode `getTotalAmount` calculant le montant total des dossiers en souffrance dans la file d'attente.

```
double getTotalAmount()
```

Q10. On souhaite pouvoir tricher de temps en temps, en insérant un dossier à une place spécifique plutôt qu'à la fin de la file pour qu'il soit traité plus vite (moyennant finance évidemment). Écrivez la méthode `cheating`. On supposera que tricher n'est possible que si la file n'est pas déjà pleine.

```
void cheating(PendingCase another, int position)
```

Q11. Réalisez un `commit` de votre travail associé au message : `"tp00-04::exo-contrôle"`.